

Assignment 2

Deadline: 20:00 (PM), May 8th Friday, 2026

Submission: Name your (scanned/e-) answer sheet by A2-{Student ID}-{Student Name}.pdf excluding both curly brackets. Send it to teaching assistant Mr. Chao WANG (QQ: 3444813826) in the course QQ group.

Part 1: Short Answer Questions (3 points each)

1. In a Raft cluster designed to tolerate $f = 3$ simultaneous replica crashes, calculate the total number of servers required in the cluster and the minimum number needed to form a majority for consensus.
2. Raft uses randomized election timeouts in the range $[T, 2T]$. If the base timeout T is set to 200 ms, what is the specific time range (in milliseconds) a follower will wait before assuming a leader has failed?
3. If a Spanner node calls `TT.now()` and receives the interval $[200 \text{ ms}, 212 \text{ ms}]$, calculate the value of ε (the worst-case clock divergence) for this interval.
4. According to the consistency hierarchy, what two conditions must a total order of operations satisfy to be considered sequentially consistent?
5. In a fault-free “Commit” execution of the 2PC protocol involving 1 coordinator and N participants, calculate the total number of messages sent by the coordinator to the participants across both the voting and decision phases.
6. Why is it required for a Raft server to store the `currentTerm` value on its hard drive rather than just in volatile memory?
7. In a causally consistent system, is it permissible for two different processes to see concurrent (non-causally related) writes in a different order?
8. In the context of distributed consensus, define the “Safety” property as it relates to decided values.
9. What specific type of RPC is used by a Raft leader to maintain its authority and prevent followers from starting new elections?

10. What are the three characteristics that make Spanner's read-only transactions "efficient" compared to its read-write transactions?

Part 2: Long-Form Analysis

1. Consider the following sequence of operations across four processes (P_A to P_D):
 - P_A : writes $x = 1$
 - P_B : reads $x = 1$, then writes $x = 2$
 - P_C : reads $x = 2$, then reads $x = 1$
 - P_D : reads $x = 1$, then reads $x = 2$

Task A: Is this sequence valid under Causal Consistency? Justify your answer by identifying the causal relationship (if any) between the write of $x = 1$ and the write of $x = 2$. (10 points)

Task B: Is this sequence valid under Sequential Consistency? Explain why or why not, focusing on whether there is a total order that respects both process order and the values read. (10 points)

2. The Spanner system maintains an invariant: if transaction T_2 starts after transaction T_1 commits, then T_2 must have a larger timestamp ($S_2 > S_1$). (10 points each)

Task A: Explain the "Commit Wait" rule. How does the leader of a transaction use $TT.after(t)$ to ensure that no client can see the effects of a transaction until the absolute time has definitely passed the commit timestamp? (10 points)

Task B: If ε (the clock uncertainty) increases significantly, how does this impact the performance of Spanner's write transactions? Reference the relationship between ε and the "commit wait" period. (10 points)

Part 3: Comparative Design Questions

1. Compare and contrast consensus models Raft vs. Viewstamped Replication. (9 points each)
 - Compare how a leader is identified in Raft versus traditional view changes. In Raft, what determines if a candidate is "best" suited to be elected leader regarding its log completeness?

- How do “Terms” in Raft function as a “Global Version Number” to neutralize old leaders, and how does this compare to the concept of “Views” in consensus protocols?
2. Compare and contrast distributed database design: Spanner vs. Eventually Consistent Systems (9 points each)
- Google’s BigTable originally used an eventually consistent approach across datacenters. Contrast the developer experience of building applications on an eventually consistent system like BigTable versus a strictly serializable system like Spanner.
 - Spanner uses 2-Phase Commit (2PC) for multi-shard transactions. Explain why Spanner can offer better performance for read-dominant workloads despite the high cost of strict serializability for writes.